

- Marco Lippi and Paolo Frasconi. Prediction of protein β -residue contacts by markov logic networks with grounding-specific weights. *Bioinformatics*, 25(18):2326–2333, 2009.
- David Maier and David S. Warren. *Computing with Logic: Logic Programming with Prolog*. Benjamin-Cummings Publishing Co., Inc., Redwood City, CA, USA, 1988.
- Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Deepproblog: Neural probabilistic logic programming. In *NeurIPS*, 2018.
- David Mascharka, Philip Tran, Ryan Soklaski, and Arjun Majumdar. Transparency by design: Closing the gap between performance and interpretability in visual reasoning. In *CVPR*, 2018.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Belle-mare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 02 2015.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.
- Stephen Muggleton. Inductive logic programming. *New Gener. Comput.*, 8(4):295–318, 1991.
- Stephen Muggleton. Stochastic logic programs. *Advances in Inductive Logic Programming*, 32: 254–264, 1996.
- Arvind Neelakantan, Quoc V Le, and Ilya Sutskever. Neural programmer: Inducing latent programs with gradient descent. In *ICLR*, 2016.
- Nils J Nilsson. *Principles of Artificial Intelligence*. Springer Science & Business Media, 1982.
- Emilio Parisotto, Abdel-rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. Neuro-symbolic program synthesis. In *ICLR*, 2017.
- Hanna M Pasula, Luke S Zettlemoyer, and Leslie Pack Kaelbling. Learning symbolic models of stochastic domains. *JAIR*, 29:309–352, 2007.
- Scott Reed and Nando De Freitas. Neural programmer-interpreters. In *ICLR*, 2016.
- Matthew Richardson and Pedro Domingos. Markov logic networks. *Mach. Learn.*, 62(1-2):107–136, 2006.
- Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In *NIPS*, 2017.
- Adam Santoro, David Raposo, David G Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia, and Tim Lillicrap. A simple neural network module for relational reasoning. In *NIPS*, 2017.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354, 2017.
- Siddharth Srivastava, Neil Immerman, and Shlomo Zilberstein. A new representation and associated algorithms for generalized planning. *Artif. Intell.*, 175(2):615–647, 2011.
- Sainbayar Sukhbaatar, arthur szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *NIPS*, 2015.
- Shao-Hua Sun, Hyeonwoo Noh, Sriram Somasundaram, and Joseph Lim. Neural program synthesis from diverse demonstration videos. In *ICML*, 2018.
- Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *NIPS*, 2014.

- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Martijn Van Otterlo. *The Logic of Adaptive Behavior*. IOS Press, 2009.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, 2017.
- Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *NIPS*, 2015.
- Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv:1502.05698*, 2015.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connect. Sci.*, 3(3):241–268, 1991.
- Jiajun Wu, Joshua B Tenenbaum, and Pushmeet Kohli. Neural scene de-rendering. In *CVPR*, 2017.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016.
- Fan Yang, Zhilin Yang, and William W Cohen. Differentiable learning of logical rules for knowledge base reasoning. In *NIPS*, 2017.
- Wenyuan Zeng, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Incorporating relation paths in neural relation extraction. In *EMNLP*, 2017.
- Yuke Zhu, Alireza Fathi, and Li Fei-Fei. Reasoning about Object Affordances in a Knowledge Base Representation. In *ECCV*, 2014.

SUPPLEMENTARY MATERIAL

This supplementary material is organized as follows. First, we provide more details for our training method and introduce the curriculum learning used for reinforcement learning tasks in Appendix A. Second, in Appendix B, we provide more implementation details and hyper-parameters of each task in Section 3. Next, we provide deferred discussion of NLM extensions in Appendix C. Besides, we give a proof of how NLMs could realize the forward chaining of a set of logic rules defined in Horn clauses. See Appendix D for details. In Appendix E, We also provide a list of sample rules for the blocks world problem in order to exhibit the complexity of describing a strategies or policies. Finally, we also provide a minimal implementation of NLM in TensorFlow for reference at the end of the supplementary material (Appendix F).

A TRAINING METHOD AND CURRICULUM LEARNING

In this section, we provide hyper-parameter details of our training method and introduce the exam-guided curriculum learning used for reinforcement learning tasks. We also provide details of the data generation method.

A.1 TRAINING METHOD

We optimize both NLM and MemNN with Adam (Kingma & Ba (2015)) and use a learning rate of $\alpha = 0.005$.

For all supervised learning tasks (i.e. family tree and general graph tasks), we use Softmax-Cross-Entropy as loss function and a training batch size of 4.

For reinforcement learning tasks (i.e. the blocks world, sorting and shortest path tasks), we use REINFORCE algorithm (Sutton & Barto (1998)) for optimization. Each training batch is composed of a single episode of play. Similar to A3C (Mnih et al. (2016)), we add policy entropy term in the objective function (proposed by Williams & Peng (1991)) to help exploration. The update function for parameters θ of policy π is

$$\Delta\theta = \alpha[v_t \nabla_{\theta} \log \pi(a_t|s_t; \theta) + \beta \nabla_{\theta} H(\pi(s_t; \theta))],$$

where H is the entropy function, s_t and a_t are the state and action at time t , v_t is the discounted reward starting from time t . The hyper-parameter β is set according to different environments and learning stages depending on the demand of exploration.

In all environments, the agent receives a reward of value 1.0 when it completes the task within a limited number of steps (which is related to the number of objects). To encourage the agent to use as few moves as possible, we give a reward of -0.01 for each move. The reward discount factor γ is 0.99 for all tasks.

Table 3: Hyper-parameters for reinforcement learning tasks. The meaning of the hyper-parameters could be found in Section A.1 and Section A.2. For the `Path` environment, the step limit is set to the actual distance between the starting point and the targeting point, to encourage the agents to find the shortest path.

Task	Range	Step Limit	β_{init}	Ω	Epochs	Train Epoch Episodes	Evaluation Episodes
Sorting	$m \in [4, 10]$	$2m$	0.01	0.5	5	200	200
Path	$m \in [3, 12]$	<i>opt</i>	0.1	0.5	40	600	3000
Blocks World	$m \in [2, 12]$	$4m$	0.2	0.6	50	1000	3000

A.2 CURRICULUM LEARNING GUIDED BY EXAMS AND FAILS

Inspired by the education system of humans, we employ an exam-guided curriculum learning (Bengio et al., 2009) approach for training Neural Logic Machines. We heuristically label each training